

设置状态

概述

在 **FSM 模式** 下，游戏本质上是状态 **state** 间的转移，因此显而易见的，制作游戏的主要工作便是各个状态的脚本。

创造你的第一个状态

1. 创建并编辑 **main_state.gd**

2. 编写状态

- 创建一个 **main_state** 函数，作为这个状态的入口：

```
func main_state():  
    TXT("era靛紫档案") # Create a text unit  
    BOX([ # Create a series of units, pack in a box  
        BTN("开始游戏", "start_game"), # Create a button unit, and set corresponding ne  
        BTN("读取存档", "load_game")  
    ])
```

你可以使用 **BOX** 来包裹一组界面显示单元，并且用这种方式调整它们的布局。对于各个UI元素，诸如 **TXT**、**BTN**、**BOX** 等，参考API文档来查看更进一步的参数选项。

- 可以注意到的是，一个按钮 **BTN** 显然会带来状态的转换，因此在它声明时也指定了它被点击时应当转向的新状态的名字。

3. 下一个状态函数 1.使用如下代码来设置下一个状态 (**start_game** 和 **load_game**):

```
func start_game():  
    TXT("你喜欢什么样的主角? ")  
    BTN("喜爱淑女的主角", "select_char_0")  
    BTN("财大气粗的主角", "select_char_1")  
    BTN("神父气质的主角", "select_char_2")  
    BTN("流氓热血的主角", "select_char_3")
```

```
func load_game():  
    TXT("读取存档")  
    BTN("读取不了，返回", "main_state")
```

```
# Continue with select_char_0, select_char_1, select_char_2, select_char_3...
```

现在，你的 **main_state** 的按钮应该可以正常工作了。如果你希望继续完成编码工作，那么 **start_game** 部分声明了更多状态，你可以继续完善这些代码。 2. 遵循这样的工作流程，你就可以逐步构建你的第一个游戏

数据文件

在 **FSM 模式** 下编写游戏的另一个重要部分是数据文件。无论是口上、角色数据、物品和装备数据，亦或是其他更多类型的信息，都需要存储为数据文件以供程序查询。

目前，ERA-Engine支持以下数据文件格式：

已经支持的格式

CSV

CSV（逗号分隔值）是一种文本数据格式，它使用逗号分隔不同的值，并使用换行符分隔记录。CSV 数据以纯文本形式存储表格数据（数字和文本），其中每一行通常代表一条数据记录。

CSV 格式最适合表示记录集或记录序列，其中每条记录都具有相同的字段列表。这对应于关系数据库中的单个关系，或者对应于典型电子表格中的数据（但不包括计算结果）。

CSV 格式是保存数据最简单的格式，也是 Emuera 游戏保存数据的方式。

计划支持的格式

INI

INI 文件是一种计算机软件的配置文件，它由纯文本构成，结构和语法以键值对的形式组织成不同的部分。

INI 文件易于阅读和解析，因此适用于不太复杂的配置文件。

Godot 原生支持 INI 格式的文件，因此我们很快也将支持这种数据文件管理方式。

XML

XML（Extensible Markup Language, 可扩展标记语言）是一种用于存储、传输和重构数据的标记语言和文件格式。它定义了一套规则，用于以既人可读又机器可读的格式对文档进行编码。

XML 已广泛应用于互联网数据交换。数百种使用 XML 语法的文档格式已被开发出来，包括 RSS、Atom、Office Open XML、OpenDocument、SVG、COLLADA 和 XHTML。

与 CSV 相比，XML 可以提供更易读、结构化的数据组织方式。

产品设定